Neural Computing and Applications
https://doi.org/10.1007/s00521-019-04359-7

ORIGINAL ARTICLE

A dynamic ensemble learning algorithm for neural networks

Kazi Md. Rokibul Alam[1] · Nazmul Siddique[2] · Hojjat Adeli[3]

Received: 24 July 2018 / Accepted: 19 July 2019

Abstract
This paper presents a novel dynamic ensemble learning (DEL) algorithm for designing ensemble of neural networks (NNs). DEL algorithm determines the size of ensemble, the number of individual NNs employing a constructive strategy, the number of hidden nodes of individual NNs employing a constructive–pruning strategy, and different training samples for individual NN's learning. For diversity, negative correlation learning has been introduced and also variation of training samples has been made for individual NNs that provide better learning from the whole training samples. The major benefits of the proposed DEL compared to existing ensemble algorithms are (1) automatic design of ensemble; (2) maintaining accuracy and diversity of NNs at the same time; and (3) minimum number of parameters to be defined by user. DEL algorithm is applied to a set of real-world classification problems such as the cancer, diabetes, heart disease, thyroid, credit card, glass, gene, horse, letter recognition, mushroom, and soybean datasets. It has been confirmed by experimental results that DEL produces dynamic NN ensembles of appropriate architecture and diversity that demonstrate good generalization ability.

Keywords Neural network ensemble · Backpropagation algorithm · Negative correlation learning · Constructive algorithms · Pruning algorithms

1 Introduction

Neural network (NN) structures have been used for knowledge representation [1], modelling [2–4], prediction [5, 6], design automation [7], classification [8, 9], identification [10], and nonlinear control [11] applications in many domains. All these applications mainly used the monolithic structure for NN. In a monolithic structure, the NN is represented by a single NN architecture for the whole task to be performed [12–14]. Scalability is a major impairment for monolithic NN for a wide range of applications. Incremental learning is also not possible as the addition of new elements to NN requires retraining of the NN with old and new data [15, 16]. An inevitable phenomenon in the retraining of NN is the catastrophic forgetting (also known as crosstalk), which was first reported by McCloskey and Cohen [17]. Two types of crosstalk phenomena can get exposed during retraining: temporal crosstalk and spatial crosstalk. In temporal crosstalk, learned knowledge is lost during retraining of a new task. In spatial crosstalk, NN cannot learn two or more tasks simultaneously [18]. Kemker et al. [19] demonstrated that catastrophic forgetting problem in incremental learning paradigm has not been resolved despite many claims and showed methods of measuring such catastrophic forgetting can be measured. A number of attempts have been made to mitigate the phenomenon such as regularization, rehearsal and pseudorehearsal, life-long learning-based dynamic combination, dual-memory models and ensemble methods [16, 20–23]. A collection or committee of individual NNs can also be advantageous for addition of a new NN to store

✉ Nazmul Siddique
nh.siddique@ulster.ac.uk

Kazi Md. Rokibul Alam
rokib@cse.kuet.ac.bd

Hojjat Adeli
adeli.1@osu.edu

[1] Department of Computer Science and Engineering, Khulna University of Engineering and Technology, Khulna 9203, Bangladesh

[2] School of Computing, Engineering and Intelligent Systems, Ulster University, Londonderry BT48 7JL, UK

[3] Departments of Neuroscience, Neurology, and Biomedical Informatics, The Ohio State University, Columbus, OH 43210, USA

Published online: 29 July 2019

new knowledge mitigating forgetting phenomena where tasks can be subdivided [24]. Instead of employing a large NN for a complex problem, the researchers are impressed by the idea of decomposing the problem into smaller subtasks leading to smaller architecture, shorter training time and increased performance [24, 25]. NN ensemble-based classifier can also improve generalization ability [25, 26]. The structure of an NN ensemble is illustrated in Fig. 1. Each NN in the ensemble (1 through n) is first trained on the training instances. The output of the ensembles is calculated from the predicted outputs, O_i, $i = 1, 2, \ldots, n$, of the individual NNs [26]. The challenge here is to design a learning algorithm for ensemble NN. The initial weights, topology of NNs, training datasets, and training algorithms also play decisive roles in the design of ensembles [23, 25]. In general, NN ensembles are designed by mostly varying these parameters.

Many algorithms similar to NN ensembles [25] have been reported in the literature such as mixer of experts [27], boosting [28] and bagging [29]. The main drawbacks of these algorithms are manual design and predefined number of neurons in the hidden layer and the number of NNs in an ensemble.

In general, ensemble and modular approaches are employed for combining NNs. The ensemble approach attempts to generate a reliable and accurate output by combining the outputs of a set of trained NNs rather than selecting the best NN, whereas the modular approach strives to have each NN as self-contained or autonomous [14, 24]. In modular approach, the problem is divided into a number of tasks. Each task is assigned to an individual NN to be accomplished. It is not possible to know the best size of NN a priori. The size of NN is defined by the number of layers and the number of neurons in each layer. Moreover, the backpropagation (BP) [30, 31] algorithm is not useful for training NN unless the topology is known. Therefore, finding the correct topology is the foremost design issue. In order to define the topology of an NN, a number of parameters such as the number of layers, the number of hidden neurons, activation functions, and degree of connectivity have to be determined. A second issue is to determine the training parameters that include the initial weights of the NN, the learning rate, acceleration term, momentum term and weight update rule. The choice of the topological and training parameters has significant impact on the training time and the performance of the NN. Unfortunately, there is no straightforward method of selecting the parameters; rather the designer has to depend on the expert knowledge or employ empirical method.

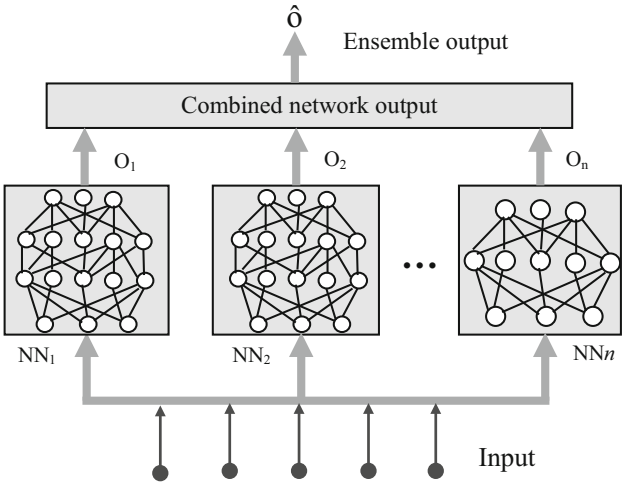

Fig. 1 A neural networks ensemble

The performance of NNs in an ensemble is dependent on a number of factors such as (1) the topology of the NNs and the initial structure; (2) the training method; (3) the learning rate; (4) the input and output representations; and (5) the content of the training sample [32]. Eventually, the numbers of NNs and the number of neurons in the hidden layers in NNs determine the performance of an ensemble. In most of the cases, these are predefined by human experts based on available a priori information. Formal learning theory is used to estimate the size of the ensemble system based on the complexity, and the examples required learning the particular function. In such cases, the generalization error becomes high if the number of examples is small. Consequently, choosing appropriate NN topology is still something of an art. The data examples play a crucial role in learning where learning is sensitive to initial weights and learning parameters [33–35].

The purpose of this research is to design an NN ensemble that addresses the following issues: (1) automatic determination of NN ensemble architecture (i.e. the number of NNs in the ensemble), (2) automatic determination of the size of individual NNs (i.e. the number of hidden neurons in individual NNs), and (3) variation of training examples for each individual NN's better learning. Real-world classification problems are used to verify the effectiveness and the generalization ability of the ensemble.

The paper is organized as follows: Sect. 2 presents the related works. Section 3 contains the description of DEL algorithm. Section 4 presents the datasets description, experimental results and comparison. Section 5 presents a discussion. Some conclusions are made in Sect. 6.

2 Related works

In ensemble learning, the individual NNs are called base learners. They are single classifiers, which are trained and combined together to ease individual errors and crop generalization independently. Hitherto, efforts have been made

to design ensemble by combining NNs based on either the accuracy or the diversity [25, 36, 37]. There are evidences that accurate and diverse NNs can produce a good ensemble that distribute errors over different regions of the input space [38, 39]. Rosen [40] proposed an ensemble learning algorithm that also trains individual NNs sequentially where the individual NNs minimize training errors as well as de-correlate previous training errors. Sequential training of an NN does not affect the NNs that were previously trained, which is a major disadvantage in ensemble learning. Consequently, there is no correlation between the errors of the individual NNs [41]. The topology of the mixtures-of-experts (ME) [27] can produce biased individual NNs which may be negatively correlated [32]. The disadvantage of ME is that it needs a separate gating NN and also can not provide a balance control over the bias–variance–covariance tradeoff [34].

A two-stage design approach is employed in most of the architectures mentioned above where individual NNs are generated first followed by combining them. As the combination stage does not provide any feedback to design stage, some individual NNs designed independently may not contribute significantly to the ensemble [34]. Therefore, some researchers proposed a one-stage design process and used a penalty term into the error function of each NN. The researchers also proposed the simultaneous and interactive training for all NNs in the ensemble instead of the independent and sequential training [41]. NNs with negative correlation can be created by reassuring specialization and cooperation among the NNs in an ensemble. This will enable NNs to learn the different regions of training data space and ensure the ensemble learns the whole data space.

To ensure interaction between NNs and simultaneous learning in an ensemble, some researchers employed evolutionary computing [32]. Liu et al. [32] applied evolutionary algorithm for ensemble learning of NNs with negative correlation. This approach can determine the optimal number of NNs and the combinations of NNs in an ensemble using fitness sharing mechanism.

Chen and Yao [33] employed multi-objective genetic algorithm [42] for regularized negative correlation learning (NCL) optimizing errors of the base NNs and their diversity in ensemble. Mousavi and Eftekhari [43] proposed static ensemble selection and deployed the popular multi-objective genetic algorithm NSGA-II [42]. This combination of static ensemble selection and NSGA-II ensures selecting the best classifiers and their optimal combination.

There are two other widely popular approaches to ensemble learning, namely constructive NN ensemble (CNNE) [44] and pruning NN ensemble (PNNE) [45]. CNNE determines the number of NNs in the ensemble and the hidden neurons of the individual NNs by employing NCL [34, 41] in an incremental fashion. On the other hand, PNNE employs a competitive decay approach. PNNE uses a neuron cooperation function in each NN for the hidden neurons and a selective deletion of NNs in the ensemble based on the criterion of over-fitting. PNNE employs NCL to ensure diversity of the NNs in the ensemble.

Islam et al. [29] proposed two incremental learning algorithms for NNs in ensemble using NCL: NegBagg and NegBoost. NegBagg fixes the number of hidden neurons of NNs in ensemble by constructive method. NegBoost also uses constructive method to fix the number of hidden neurons of NNs as well as the number of NNs in the ensemble.

Yin et al. [46] proposed a two-stage hierarchical approach to ensemble learning called dynamic ensemble of ensembles (DE^2). DE^2 comprises component classifiers and interim ensembles. The final DE^2 is obtained by weighted averaging. Cruz et al. [47] used a two-phase dynamic ensemble selection (DES) framework. In the first phase, DES extracts meta-features from training data. In the second phase, DES uses a meta-classifier to estimate the competence of the base classifier to be added to the ensemble.

Chen and Yao [48] show that NCL considers the entire ensemble as a single machine with the objective of minimizing the mean square error (MSE) and NCL does not employ regularization while training. They proposed a regularized NCL (RNCL) incorporating a regularization term for the ensemble which enables the RNCL decomposing the training objectives into sub-objectives each of which is implemented by an individual NN. RNCL shows improved performance over the NCL even when noise level is higher in datasets.

Semi-supervised learning is the mechanism of learning using a large amount of unlabelled data and a small amount of labelled data. Chen and Wang [49] proposed a semi-supervised boosting framework taking three assumptions such as smoothness, cluster and manifold into consideration where they used a cost function comprising the margin cost on labelled data and the regularization penalty on unlabelled data. Experiments on benchmarks and real-world classification reveal constant improvement by the algorithm. Semi-supervised learning is a widely popular method due to its higher accuracy at a lower effort.

The generalization of an ensemble is related to the accuracy of the base NNs and the diversity among NNs [37, 38]. Higher accuracy for the base NNs leads to the lower diversity among them. To strike a balance of the dilemma between accuracy and diversity in an ensemble, Chen et al. [50] proposed a semi-supervised NCL (SemiNCL) where a correlation penalty term on labelled and unlabeled data is incorporated into the cost function of each individual NNs in the ensemble.

Though the semi-supervised learning has been very successful for labelled and unlabelled data, its generalization ability is sensitive to incorrect labelled data. To mitigate this limitation, Soares et al. [51] proposed a cluster-based boosting (CBoost) with cluster regularization. In CBoost, the base NNs in the ensemble jointly perform a cluster-based semi-supervised optimization. Extensive experimentation shows that the CBoost has significant generalization ability over the other ensembles.

Recently, Rafiei and Adeli [52] reported a new neural dynamic classification algorithm. A comprehensive review of multiple classifier systems based on the dynamic selection of classifiers was reported by Britto et al. [53]. Recent developments in ensemble methods are analysed by Ren et al. [54]. Cruz et al. [55] reported a review on the recent advances on dynamic classifier selection techniques. Dynamic mechanism is used in the generalization phase in those studies, while the dynamic mechanism is employed in the training phase in DEL.

3 Dynamic ensemble learning (DEL)

3.1 Main steps of the algorithm

Unlike fixed ensemble architecture, DEL automatically determines the number of base learner NNs and their architectures in an ensemble during the training phase. The DEL algorithm is presented in 8 steps in the sequel. The flow diagram of the DEL algorithm is shown in Fig. 2.

Step 1 Create an ensemble with minimum architecture comprising two NNs. Each NN consists of an input layer, two hidden layers, and an output layer. The number of neurons in the input and output layers is determined by the system. Next, apply a constructive algorithm [56] based on Ash's [57] dynamic node creation method for the first (later on the odd number of NNs in sequence in the ensemble) NN training. Initially, this NN starts with a small architecture containing one node in each hidden layer. For the second (later on even number of NNs in sequence in the ensemble) NN training, apply Reed's pruning algorithm [58]. In the pruning phase of NN training, the number of neurons in the hidden layer is larger than necessary (i.e. it starts with a bulky architecture). Initialize the connection weights for each NN randomly within a small interval.

Step 2 Create separate training examples for each NN of the ensemble. In general, subsets of training examples for individual NNs are created by randomly picking from the main set of the training examples. In this work, training sets are created in such a way that if one NN learns from training examples from the first to the last, other NN learns from the last to the first of the same training examples.

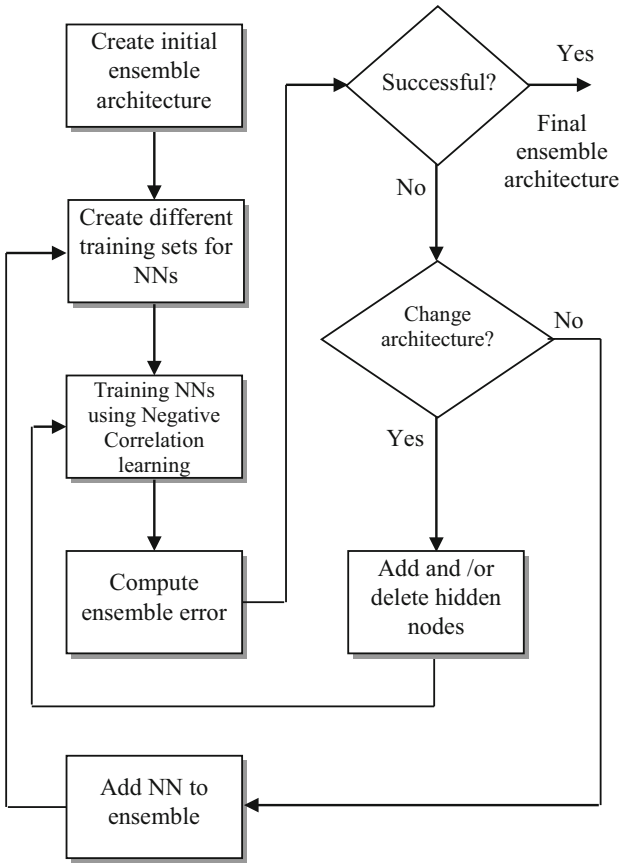

Fig. 2 Flow diagram of the DEL algorithm

Step 3 Train the NNs in the ensemble partially on the examples for a fixed number of epochs specified by the user using NCL [34, 41] regardless of whether the NNs converge or not [59].

Step 4 Compute the training error E_i for the ith NN in the ensemble according to the following rule:

$$E_i = 100\frac{O_{\max} - O_{\min}}{N.S}\sum_{n=1}^{N}\sum_{S=1}^{S}\left[(d(n,s) - F_i(n,s))^2 + \lambda P_i(n,s)\right] \quad (1)$$

where $O_{\max}$ is the maximum value and $O_{\min}$ is the minimum value of the target outputs, respectively, N is the total number of examples, S is the number of output neurons, $d(n, s)$ is the desired output, and $F_i(n, s)$ is the actual output of the neuron s in the nth training data. The rule in Eq. (1) is a combination of the rule proposed by Reed [58] and NCL for an NN error. The error E_i is independent of the size of the training examples and the number of output neurons.

Step 5 Compute the ensemble error E where E is the average of E_i of the base learner NNs. If E is small and acceptable, the ensemble architecture is believed to have the highest generalization ability and output the final ensemble. If E is not

acceptable, then either the ensemble architecture or the individual base learner NNs undergo change.

Step 6 Check the neuron addition and/or deletion criterion of individual NNs. In this criterion, hidden neurons are added or deleted if the error of individual NNs does not change after a specified number of epochs chosen by the user (see Sect. 3.2). If the criterion is not met, then the individual NNs are not good enough and the ensemble undergoes addition of new learner NN.

Step 7 Add and/or delete hidden neurons to/from the NNs to meet the addition and/or deletion criterion (see Sect. 3.2) and continue training using NCL.

Step 8 Add a new NN to the ensemble (see Sect. 3.3) if the previous NN addition improves the performance of the ensemble. Initialize and create different training sets for the new NN as in step 2. Go to step 3 for further training of the ensemble.

The above-mentioned procedure (steps 1–8) is implemented in DEL that determines the architecture of ensemble. For example, the networks in Fig. 1 work as follows: network 1 has 2 hidden layers, uses constructive algorithm for node addition, and trains examples from first to last. On the contrary, network 2 has 2 hidden layers, uses pruning algorithm for node deletion, and trains examples from last to first. Then, network 3 has a single hidden layer, uses constructive algorithm for node addition, and trains using examples from first to last. Similarly, network 4 has a single hidden layer, uses pruning algorithm for node deletion, and trains using examples from last to first and so on. The idea of varying the training examples is to enable the NNs to learn different regions of the data distribution. Major components of DEL are the addition/deletion of hidden neurons to/from learners NNs and addition of NN to ensemble described in Sects. 3.2–3.4.

3.2 Nodes addition/deletion to/from individual NNs

Both constructive and pruning algorithms provide some benefits as well as some drawbacks. At the training period of individual NNs, there may be some portions which may be critical or stable either for constructive or pruning algorithms. If all the NNs in the ensemble learn either only by constructive or only by pruning algorithm, then their learning will be very similar.

Even though NCL forces the NNs to learn from different regions of the data space, the learning will not be perfect if the NNs in the ensemble have the same architecture. Different architectures of the NNs in the ensemble will provide a different weight on the accuracy and diversity, which justifies the deployment of the hybrid 'constructive–pruning strategy' in DEL.

3.3 NN addition to the ensemble

In DEL, constructive algorithm is used to add NNs in the ensemble. New NNs are added to the ensemble if the previous addition improves the performance of the ensemble. This addition process continues until the minimum ensemble error criterion has been met.

3.4 Different training sets for individual NNs

Varying the examples into different training sets enables efficient learning and can help the ensemble learning from the whole training examples. Training sets are varied by maintaining one important criterion, i.e. training sets should have appropriate number of examples so that individual NNs obtain the necessary information for learning.

In DEL, if the first NN in the ensemble learns from odd-positioned training examples, the second one learns from even-positioned training examples, and the third one learns from other training examples in a similar fashion. In some cases, subsets of training examples are created just by partitioning or by randomly selecting. The pseudocode of DEL algorithm is shown in Algorithm 1.

Algorithm 1: DEL algorithm	
Step 1: Create ensemble with minimum architecture	
1.	Create an ensemble comprising 2 NNs with minimum architecture of 1 input-2 hidden-1 output layers
2.	Number of neurons in input and output layer is determined by the system
3.	Apply Ash's constructive algorithm for dynamic node creation for the first NN training
4.	Apply Reed's pruning algorithm for the second NN training
Step 2: Create training examples	
1	Create separate training examples for each NN
Step 3: Training NNs in ensemble	
1.	Train NNs partially for fixed number of epochs using NCL
Step 4: Compute training error	
1.	Compute the training error E_i for the ith NN using Eq. (1)
Step 5: Compute ensemble error	
1.	Compute the ensemble error E
2.	**If** $E <$ *acceptable*
3.	Output final ensemble
	Endif
Step 6: Check node addition/deletion criterion	
1.	**If** (addition/deletion criterion is not met)
2.	Add NN to ensemble
3.	Go to **Step 2**
4.	**Else**
5.	Add/delete hidden nodes to NN
6.	Go to **Step 3**
7.	**Endif**

4 Experimental analysis

The effectiveness and performance of DEL are verified on real-world benchmark problems. The datasets of the selected benchmark problems are taken from the UCI machine learning repository [60].

Different tests were carried out on DEL algorithm with varying parameter settings. For setting the correlation strength parameter λ to nonzero, the DEL performs as described in Sect. 3. For the correlation strength parameter λ equal to zero, it is the individual NN's independent training. The independent training is performed using standard backpropagation algorithm [30].

The learning rate and correlation strength parameter λ were chosen between [0.05, 1.0] and [0.1, 1.0], respectively. The initial weights for NNs were randomly generated within the interval of $[-0.5, 0.5]$. The winner-takes-all method of classification is used. Both the majority voting method and the simple averaging method are used for computing the generalization ability of the DEL. Medical and non-medical datasets described in Sects. 3.1 and 3.2 are used in the experimentation. Table 1 shows the summary of benchmark datasets.

4.1 Medical datasets

The medical datasets comprise four datasets from medical domain: the cancer, the diabetes, the heart disease, and the thyroid dataset. These datasets have some characteristics in common:

- DEL uses the similar input attributes that an expert uses for diagnosis.
- The datasets pose a classification problem, which the DEL has to classify to a number of classes or predict a set of quantities.
- Acquisition of examples from human subjects is expensive, which results in small datasets for training.
- Very often the datasets have missing values of attributes and contain a small sample of noisy data [59], which make the classification or prediction challenging.

4.1.1 The breast cancer dataset

The breast cancer dataset comprises 699 examples. 458 examples are benign, and 241 examples are malignant. There are 9 attributes of a tumour collected from expensive microscopic examinations. The attributes relate to the thickness of clumps, the uniformity of cell size and shape, the amount of marginal adhesion, and the frequency of bare nuclei. The problem is to classify the tumour as either benign or malignant.

4.1.2 The diabetes dataset

The diabetes dataset comprises 768 examples of which 500 belong to class 1 and 268 belong to class 2. Datasets are collected from female patients of 21 years of age or older and of Pima Indian heritage. There are 8 attributes to be classified as either 'tested positive for diabetes' or 'tested not positive for diabetes'.

4.1.3 The heart disease dataset

The heart disease datasets comprise 920 examples. The datasets are collected from expensive medical tests on patients. There are 35 attributes to be classified as presence or absence of heart disease.

4.1.4 The thyroid dataset

The thyroid disease dataset comprises 7200 examples collected from patients through clinical tests. There are 21 attributes to be classified in three classes, i.e. normal, hyper-function and subnormal function. 92% of the

Table 1 Summary of benchmark datasets

Dataset	No. of examples	Attributes	Classes	Training set	Test set
Cancer	699	9	2	349	175
Diabetes	768	8	2	384	192
Heart	920	35	2	460	230
Thyroid	7200	21	3	3600	1800
Credit C	690	51	2	345	172
Glass	214	9	6	107	53
Gene	3175	120	3	1588	793
Horse	364	58	3	182	91
Letter	20,000	16	26	16,000	4000
Mushroom	8124	125	2	4062	2031
Soybean	683	82	19	342	171

patients are normal, which insists that the classifier accuracy must be significantly higher than 92%.

4.2 Non-medical datasets

The non-medical datasets comprise seven datasets from different other domains: the credit card, glass, gene, horse, letter, mushroom, and soybean datasets.

4.2.1 The credit card dataset

The credit card dataset comprises 690 examples collected from real credit card applications by customers with a good mix of numerical and categorical attributes. There are 51 attributes to be classified as credit card granted or not granted by the bank. 44% of the examples in the datasets are positive. The datasets also contain 5% missing values in the examples.

4.2.2 The glass dataset

The classification of glass dataset is used for forensic investigations. The datasets comprise 214 examples collected from chemical analysis of glass splinters. There are 70, 76, 17, 13, and 19 examples for 6 classes, respectively. The datasets contain 9 attributes of continuous value to be classified into 6 classes.

4.2.3 The gene dataset

The gene dataset comprises 3175 examples of intron/exon boundaries of DNA sequences elements or nucleotide. A nucleotide is a four-valued nominal attribute and encoded binary, i.e. $\{-1, 1\}$. There are 120 attributes to be classified into three classes: exon/intron (EI) boundary, intron/exon (IE) boundary, or none of these. EI boundary is called donor, and IE boundary is called acceptor. 25% examples of the dataset are donors, and 25% examples are acceptors.

4.2.4 The horse dataset

The horse dataset comprises 364 examples of horse colic. Colic is an abdominal pain in horses, which can result in death. There are 58 attributes collected from veterinary examination to be classified into three classes: horse will survive, die, or euthanized. The dataset contains 62% examples of survival, 24% examples of death, and 14% examples of euthanized. About 30% of the values in the dataset are missing, which poses challenges in classification.

4.2.5 The letter recognition dataset

Alphabet consists of 26 letters, and recognition of letters is a large classification problem. It is a tough benchmark problem for the DEL algorithm. The dataset contains 20,000 examples of digitized patterns. Each example was converted into 16 numerical attributes (i.e. real valued vector), which are to be classified into 26 classes.

4.2.6 The mushroom dataset

The mushroom dataset comprises 8124 examples based on hypothetical observations of mushroom species described in a book. There are 125 attributes of the mushrooms collected based on the shape, colour, odour, and habitat. 30% of the examples have one missing attribute value. 48% of examples are poisonous. The classifier has to categorize the mushrooms as edible or poisonous.

4.2.7 The soybean dataset

The soybean dataset comprises 683 examples collected from the descriptions of beans. The attributes are based on the normal size and colour of leaf, the size of spots on leaf, hallow spots, normal growth of plant, the rooted roots, and the plant's life history, treatment of seeds, and the air temperature. There are 82 attributes to be classified into 19 diseases of soybeans. There are missing values of attributes in most of the examples.

4.3 Experimental setup

Datasets are divided into training and testing sets, and no validation set is used in the experimentation. The classification error rate is calculated according to:

$$C_i = 100 * \frac{\text{T.T.P} - \text{C.P}}{\text{T.T.P}} \tag{2}$$

where T.T.P denotes the total number of test patterns and C.P denotes the total number of correctly classified patterns. The numbers of examples in the training and test sets are chosen based on the reported works in the literature so that a comparison of results is possible. The size of the training and testing sets used in DEL is shown in Table 1.

4.4 Experimental results

A summary of the experimental results of the DEL algorithm carried on 11 datasets described in Sects. 4.1 and 4.2 is presented in Table 2. The classification error is defined as the percentage of wrong classifications in the test set defined by Eq. 2. Table 3 shows the comparison of DEL with its component individual networks in terms of

Table 2 Results obtained applying the proposed learning model for 11 benchmark datasets

Dataset	Ensemble		Epoch	Error	
	Initial	Final		Training	Classification
Cancer	9-4-2-2 9-12-2-2	(9-8-2-2), (9-10-2-2), (9-8-2), (9-10-2)	113	0.01	0.571
Diabetes	8-4-4-2 8-9-4-2	(8-8-4-2), (8-7-4-2), (8-8-2), (8-7-2), (8-8-2), (8-7-2)	212	5.00	22.917
Heart disease	35-9-2-2 35-13-2-2	(35-12-2-2), (35-11-2-2), (35-12-2), (35-11-2), (35-10-2)	85	4.00	15.652
Thyroid	21-8-3-3 21-18-2-3	(21-12-3-3), (21-16-2-3), (21-12-3), (21-16-3), (21-12-3), (21-16-3)	400	0.71	4.444
Credit card	51-10-2-2 51-28-2-2	(51-13-2-2), (51-26-2-2), (51-13-2), (51-26-2), (51-13-2)	350	0.77	12.209
Glass	9-5-7-6 9-11-7-6	(9-10-7-6), (9-9-7-6), (9-10-6), (9-9-6)	300	3.74	26.415
Gene	120-14-5-3 120-18-6-3	(120-17-5-3), (120-16-6-3), (120-18-3), (120-16-3)	275	0.05	10.971
Horse	58-13-2-3 58-18-2-3	(58-17-2-3), (58-16-2-3), (58-17-3), (58-16-3), (58-17-3)	350	6.50	23.077
Letter recognition	16-20-23-26 16-24-23-26	(16-23-23-26), (16-22-23-26), (16-23-26), (16-22-26)	215	0.004	12.2
Mushroom	125-1-2-2 125-7-2-2	(125-4-2-2), (125-6-2-2), (125-4-2)	95	0.002	0.591
Soybean	82-22-7-19 82-26-8-19	(82-25-7-19), (82-24-8-19), (82-25-19), (82-24-19), (82-25-19)	261	0.0006	4.094

Table 3 Comparison of ensemble's classification error with its component NNs for the glass database

Ensemble/NN	Architecture	Classification error
Ensemble	(9-10-7-6), (9-9-7-6), (9-11-6), (9-9-6)	26.415
NN1	9-10-7-6	30.189
NN2	9-9-7-6	37.736
NN3	9-11-6	32.075
NN4	9-9-6	32.075

classification error rates for glass dataset. It shows the error rates for glass datasets are relatively higher than the other datasets. This is due to the error rates of the individual NNs that led to the higher error rate of the ensemble. Table 4a shows the accuracy of NNs and the common intersection and the diversity of the NNs of ensemble for the glass dataset is shown in Table 4b. The accuracy Ω means the correct response sets of the individual NNs, whereas the diversity ς means the number of different examples correctly classified by individual NNs. If S_i is the correct response set of the ith NN in the testing set, Ω_i is the size of S_i, and $\Omega_{i1,i2,\ldots,ik}$ is the size of the set $S_{i1} \cap S_{i2}, \ldots, \cap S_{ik}$, then the diversity ς of the ensemble is $\Omega_i = \Omega_{i1,i2,\ldots,ik}$. For the glass dataset, DEL produced an ensemble of four NNs

Table 4 For the test datasets of glass problem: (a) the accuracy and intersection of NNs; (b) the measure of diversity of these individual NNs [61]

(a) *Accuracy of NNs*			
$\Omega_1 = 37$	$\Omega_2 = 33$	$\Omega_3 = 36$	$\Omega_4 = 36$
$\Omega_{12} = 33$	$\Omega_{13} = 34$	$\Omega_{14} = 34$	$\Omega_{23} = 30$
$\Omega_{24} = 30$	$\Omega_{34} = 34$	$\Omega_{123} = 30$	$\Omega_{124} = 30$
$\Omega_{134} = 33$	$\Omega_{234} = 30$	$\Omega_{1234} = 29$	
(b) *Diversity of NNs in ensemble*			
$\varsigma_{12} = 4$	$\varsigma_{13} = 5$	$\varsigma_{14} = 5$	$\varsigma_{23} = 9$
$\varsigma_{24} = 9$	$\varsigma_{34} = 4$	$\varsigma_{123} = 7$	$\varsigma_{124} = 7$
$\varsigma_{134} = 5$	$\varsigma_{234} = 5$	$\varsigma_{1234} = 9$	

(N_1, N_2, N_3, and N_4). The sizes of the correct responses are $S_1 = 37$, $S_2 = 33$, $S_3 = 36$, and $S_4 = 36$. The large variations in accuracies are caused by the incremental learning used by DEL. The ensemble started with N_1 and N_2 and trained them. When the two failed to achieve a successful ensemble, DEL added N_3 and N_4 at a final step. The size of $S_1 \cap S_2 \cap S_3 \cap S_4$ was only 29 resulting in diversity $\varsigma = 8$ among N_1, N_2, N_3, and N_4.

It is demonstrated here that the DEL uses a smaller number of training cycles to find the dynamic ensemble architecture with a small classification error. For example, for the glass dataset DEL with dynamic architecture produces a final ensemble with only four individual networks. Only five hidden nodes were added to individual networks training with constructive algorithm, and two hidden nodes were deleted from individual networks' while training with a pruning algorithm. DEL achieved a classification error of 26.415% for this dataset. According to the comparison with other algorithms shown in Table 5, DEL achieves the lowest percentage of classification error.

To demonstrate how a hidden neuron's output changes during the entire training period, the hidden neurons' output for the cancer dataset is shown in Fig. 3. Constructive algorithm was used for training one network. The individual network started the training with one node in its first hidden layer and two nodes in its second hidden layer. During the training period, four nodes were added to the first hidden layer of the network and nodes in second hidden layer were kept fixed at two nodes. The outputs stabilize and the convergence curve becomes smooth after about 100 iterations, indicating that the learning may not require a very large number of iterations.

Figure 4 shows the training error profile of ensemble for cancer, heart disease, glass, and soybean datasets. Two from medical and two from non-medical datasets are chosen. During the intermediate period of the training, individual networks were added to the ensemble by constructive strategy and hidden nodes were added as well as deleted from corresponding individual networks using a hybrid constructive–pruning strategy. For example, for cancer dataset in Fig. 4, the ensemble started with two individual networks with architecture (9-4-2-2) and (9-12-2-2). The NN architecture (9-4-2-2) has 9 inputs, two hidden layers with 4 and 2 neurons, respectively, and 2 outputs. The NN architecture (9-12-2-2) has 9 inputs, two hidden layers with 12 and 2 neurons, respectively, and 2 outputs. Constructive algorithms for individual network (9-4-2-2) and pruning algorithm for individual network (9-12-2-2) were applied during training. During the training, individual NNs with architectures (9-4-2) and (9-12-2) were added to the ensemble. Hidden nodes were added to individual networks (9-4-2-2) and (9-4-2) as constructive algorithms were used to train them. Hidden nodes were deleted from individual networks (9-12-2-2) and (9-4-2) as these two were trained using pruning algorithm. After addition of individual networks and hidden nodes by constructive strategy and deletion of hidden nodes by pruning strategy, the final ensemble with individual NN architectures of (9-8-2-2), (9-10-2-2), (9-8-2), and (9-10-2) was attained.

Figure 5a, b shows the training error profiles of individual NNs with constructive algorithm. For example, Fig. 5a shows the curves of individual networks for which constructive algorithms were applied starting with architectures (9-4-2-2) (indicated by solid line) and (9-4-2) (indicated by dash line) for cancer dataset. At the intermediate period of training, hidden nodes were added to individual networks by the dynamic node creation (DNC) method until this node addition increased the performance of the ensemble. Finally, all these constructive networks in the ensemble completed training with (9-8-2-2) and (9-8-2) architectures. Solid lines indicate NNs with 2 hidden layers, and dash lines indicate NN with single hidden layer from Figs. 5, 6, 7 and 8.

Figure 6a, b shows training error profiles of individual NNs with pruning algorithm. The pruning algorithm has an impact on error profiles which is visible from the non-smooth curves. Figure 6a shows the curves of individual NNs applied to cancer dataset starting with (9-12-2-2) and (9-12-2) architectures. At the intermediate training period, hidden nodes were deleted from individual networks by the sensitivity calculation method until this node deletion increased the performance of the ensemble. Finally, all these pruning networks in the ensemble ended up training with (9-10-2-2) and (9-10-2) architectures.

Figure 7a, b shows the curves of hidden nodes addition to the individual NNs training applying constructive algorithm. In this case, individual networks with small architecture started training and at the intermediate training period, hidden nodes were added to the first hidden layer of the individual network sensitivity by the dynamic node creation method. For example, Fig. 7a shows the curves of the hidden nodes addition to individual networks trained using constructive algorithm for cancer dataset. Here, the individual network started training with (9-4-2-2) and (9-4-2) architectures and finally ended up training with (9-8-2-2) and (9-8-2) architectures.

Figure 8a, b shows the curves of the hidden nodes deletion from the individual NNs training applying pruning algorithm. Individual networks with architecture larger than necessary started training in this case and at the intermediate training period, hidden nodes that deem not necessary were deleted from the first hidden layer of the individual network by the sensitivity calculation method.

Hidden node with the lowest sensitivity was deleted. If the deleted node does not possess the lowest sensitivity,

Table 5 Test set error rates for the datasets: comparison of DEL with results of (1) a single NN classifier (Stan); (2) an ensemble created by varying random initial weights (Simp); (3) an ensemble created by Bagging method; (4) an ensemble created by Arcing method, (5) an ensemble created by Ada method [63], (6) a semi-supervised ensemble learning algorithm, i.e. SemiNCL [50], and (7) a cluster-based boosting (CBoost) ensemble in two versions, i.e. CBoost-Sup and CBoost-Semi [51]

Dataset	DEL	Stan	Simp	Bag	Arc	Ada	(with % of labelled data)								
							SemiNCL [45]			CBoost-Sup [46]			CBoost-Semi [46]		
							5%	10%	20%	5%	10%	20%	5%	10%	20%
Cancer	0.571	3.4	3.5	3.4	3.8	4.0	–	–	–	–	–	–	–	–	–
Diabetes	22.917	23.9	23.0	22.8	24.4	23.3	–	–	–	–	–	–	–	–	–
Heart d	15.562	18.6	17.4	17.0	20.7	21.1	–	–	–	–	–	–	–	–	–
Thyroid	4.444	–	–	–	–	–	–	–	–	–	–	–	–	–	–
Credit card	12.209	14.8	13.7	13.8	15.8	15.7	15.87 ± 3.73	14.68 ± 2.84	14.10 ± 2.58	21.47 ± 3.47	19.96 ± 2.67	18.35 ± 3.03	18.67 ± 1.26	16.18 ± 2.73	15.82 ± 3.44
Glass	26.415	38.6	35.2	33.1	32.0	31.1	–	–	–	38.96 ± 12.01	19.01 ± 7.31	18.84 ± 6.88	36.31 ± 10.36	19.54 ± 4.57	20.32 ± 7.66
Gene	10.971	–	–	–	–	–	–	–	–	–	–	–	–	–	–
Horse	23.077	–	–	–	–	–	36.59 ± 6.30	33.82 ± 4.97	32.80 ± 5.22	25.87 ± 6.27	23.45 ± 5.23	29.11 ± 4.99	26.23 ± 6.17	22.52 ± 5.19	29.12 ± 5.04
Letter	12.2	18.0	12.8	10.5	5.7	6.3	–	–	–	–	–	–	–	–	–
Mushroom	0.591	–	–	–	–	–	–	–	–	–	–	–	–	–	–
Soybean	4.094	9.2	6.7	6.9	6.7	6.3	–	–	–	–	–	–	–	–	–

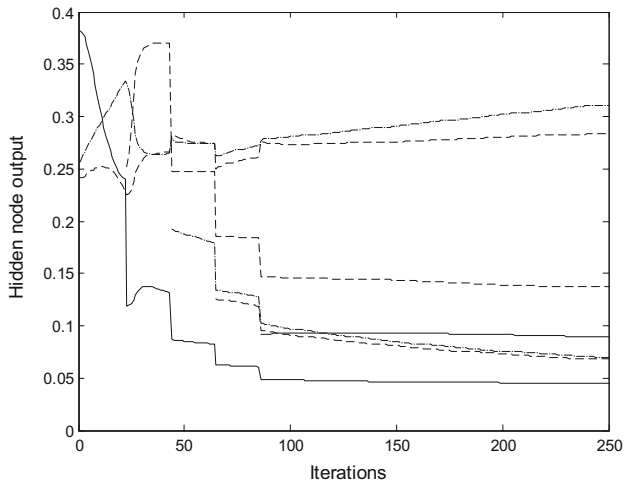

Fig. 3 The hidden nodes output of a network with initial architecture (9-4-2-2) and final architecture (9-8-2-2) for the cancer datasets

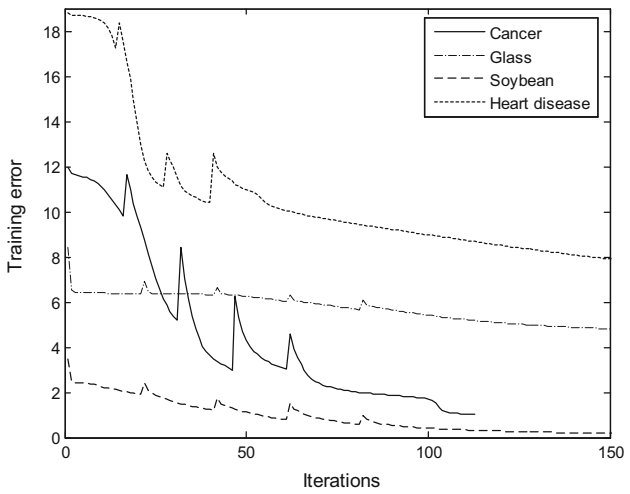

Fig. 4 The error profile of the ensemble: cancer dataset; glass dataset; soybean dataset; and heart disease dataset

then the weights were restored. For example, Fig. 8a shows the curves of hidden nodes deletion from individual networks training by pruning algorithm for cancer dataset. Individual networks here started training with (9-12-2-2) and (9-12-2) architectures and finally completed training with (9-10-2-2) and (9-10-2) architectures.

Figure 9a, b shows the curves of individual networks addition during the training period. Individual networks were added to the ensemble applying constructive strategy. Initially, the number of NNs in the ensemble was two. When addition increased the performance of the ensemble, the number was increased. For example, Fig. 9a shows the curve of individual network addition to the ensemble for cancer dataset. The curve shows that network addition to the ensemble completed training with four networks.

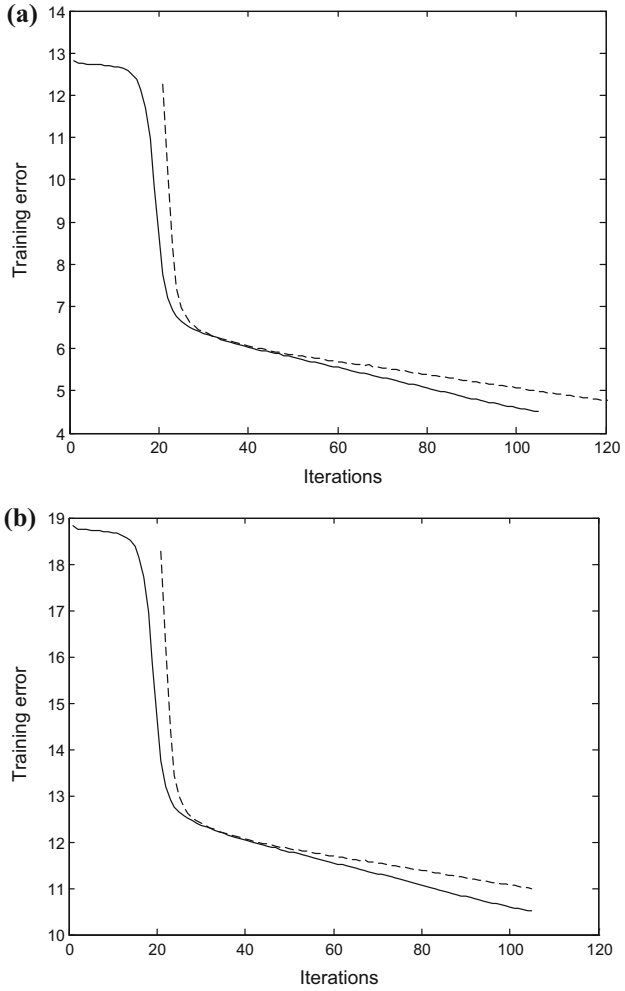

Fig. 5 The error of the individual networks for constructive algorithm: **a** for the cancer dataset; **b** for the heart disease dataset. Solid line indicates NN with 2 hidden layers, and dash line indicates NN with single hidden layer (shown in Table 2)

4.5 Correlations among the individual NNs

In Tables 6, 7, and 8, Cor_{ij} means correlation between individual networks j and i in the ensemble.

The distinguishable difference between Tables 6 and 7 is the negative correlation strength parameter $\lambda = 0.2$, so that the correlation between any two networks is positive in Table 6. But in Table 7, the negative strength correlation parameter is $\lambda = 1.0$ so that in almost all cases the value of correlation between any two networks is negative.

The distinguishable difference between Tables 8 and 9 is the negative correlation strength parameter $\lambda = 0.2$ in Table 8 so that the correlation between any two networks is positive. But in the case of Table 9, the negative correlation strength parameter is $\lambda = 1.0$, which results in negative correlation between any two networks in many cases.

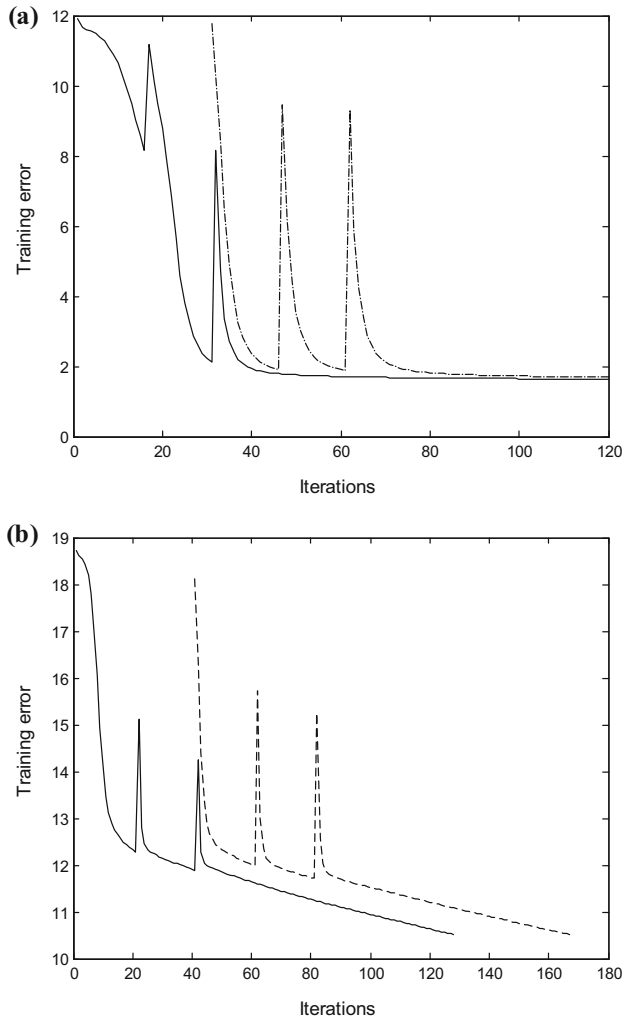

Fig. 6 The error of the individual networks for pruning algorithm: **a** for the cancer dataset; **b** for the heart disease dataset. Solid line indicates NN with 2 hidden layers and dash line indicates NN with single hidden layer (shown in Table 2)

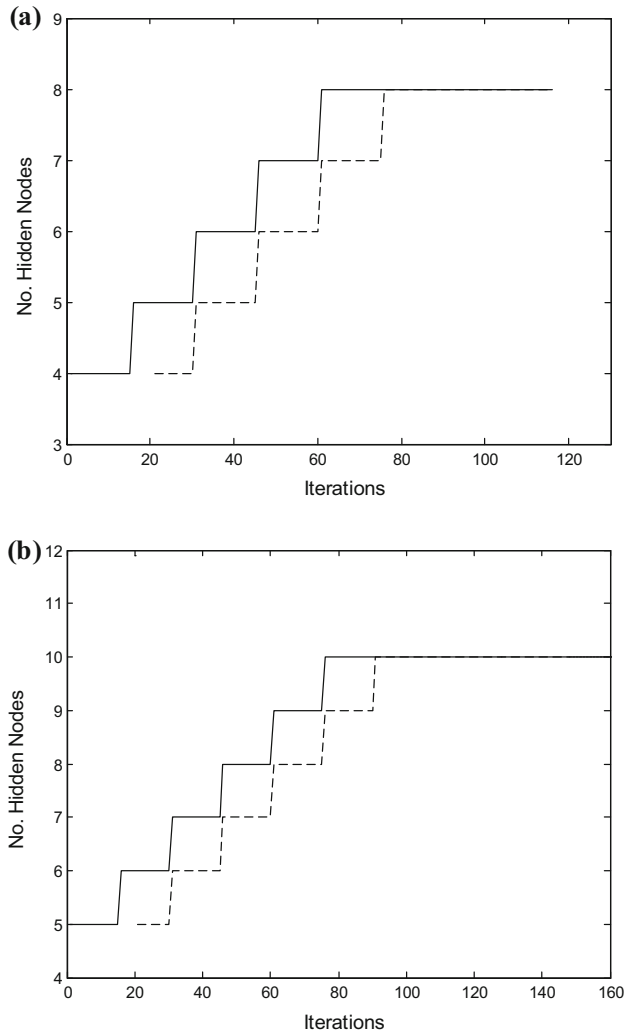

Fig. 7 Hidden nodes addition for individual networks training with constructive algorithm: **a** for the cancer dataset; **b** for the glass dataset. Solid line indicates NN with 2 hidden layers, and dash line indicates NN with single hidden layer (shown in Table 2)

4.6 Comparison

To verify the performance of DEL algorithm, the results are compared with popular empirical study of ensemble network by Opitz and Maclin [62], a semi-supervised ensemble learning algorithm, i.e. SemiNCL by Chen et al. [50], and a fully semi-supervised ensemble approach to multiclass semi-supervised classification in two versions, i.e. CBoost-Sup and CBoost-Semi by Soares et al. [51]. Opitz and Maclin have studied a number of networks such as a simple NN, an ensemble with varying initial weights, Bagging ensemble, and Boosting ensemble. They used resampling based on Arcing and Ada method. A confidence level of 95% can be achieved by an ensemble method than a single-component classifier [34]. Opitz and Maclin didn't apply thyroid, gene, horse, and mushroom datasets in their experiments; therefore, the results are not available for comparison and marked as '−' in the table. Chen et al. [50] and Soares et al. [51] both have presented test errors by mean ± standard deviation % with 5%, 10%, and 20% of labelled data. They also didn't apply cancer, diabetics, heart, thyroid, gene, letter, mushroom and soybean datasets in their experiments; therefore, the results are not available for comparison and marked as '−' in the table.

5 Discussions

Most of the existing ensemble learning methods use trail-and-error method to determine the number and architecture of NNs in the ensemble. Most of them use a two-stage design process for designing an ensemble. In the first stage, individual NNs are created, and in the second stage these NNs are combined. In the ensemble, the number of NNs

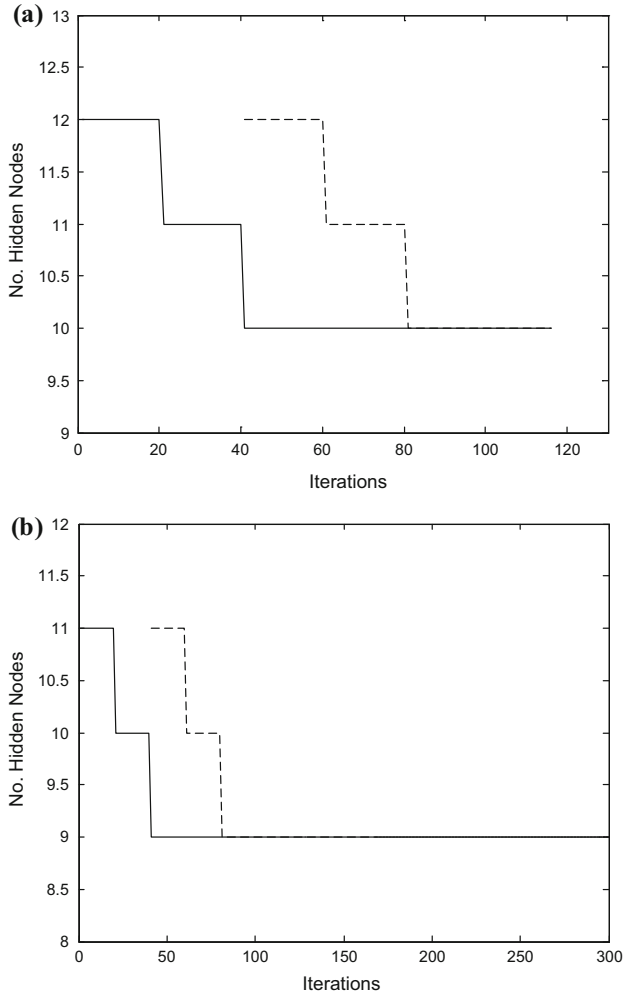

Fig. 8 Hidden nodes deletion for individual networks training with constructive algorithm: **a** for the cancer dataset; **b** for the glass dataset. Solid line indicates NN with 2 hidden layers, and dash line indicates NN with single hidden layer (shown in Table 2)

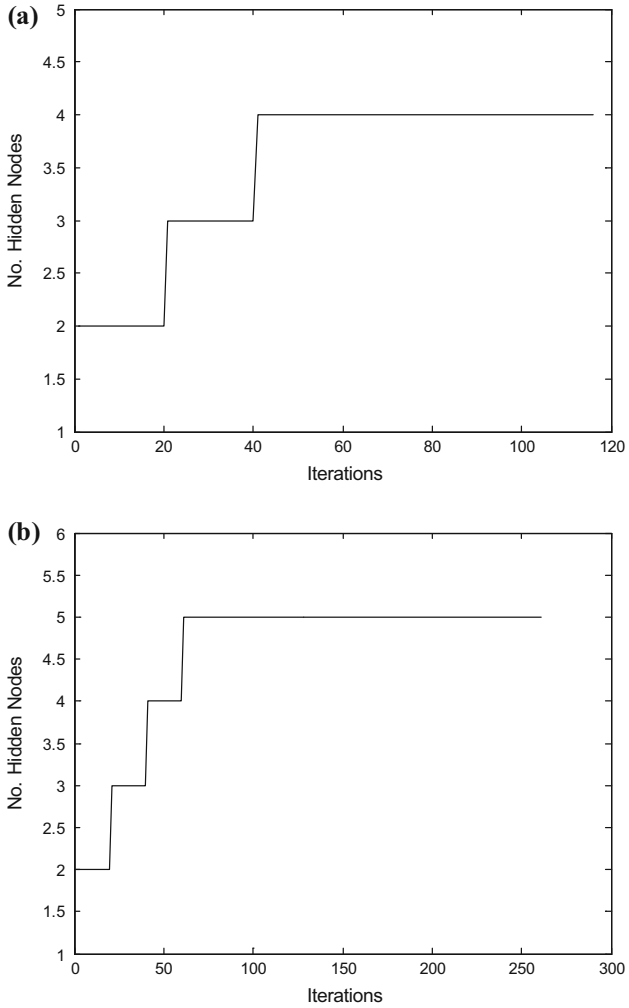

Fig. 9 Individual network addition to the ensemble at the training period: **a** for the cancer dataset; **b** for the soybean dataset

Table 6 Correlation of networks for the cancer dataset for $\eta = 0.1$, $\lambda = 0.2$. In this case, iteration continued 116. In ensemble, individual networks required is 4

$Cor_{12} = 0.018309$	$Cor_{13} = 0.021606$	$Cor_{14} = 0.019284$
$Cor_{23} = 0.017636$	$Cor_{24} = 0.015741$	$Cor_{34} = 0.018576$

and the number of hidden neurons in the individual networks are predefined and fixed. These existing methods use two cost functions for designing the ensemble. One is for the accuracy, and another is for diversity. In most of the existing ensemble methods, individual NNs are trained independently or sequentially rather than simultaneously, which lead to loss of interaction among NNs in the ensemble. In ensemble training, the previously trained network is not affected.

In DEL, we presented a dynamic approach to determine the topology of an ensemble. This dynamic approach determines the number and architecture of the individual NNs in the ensemble. Such a dynamic approach is entirely new to designing NN ensemble. In DEL, better diversity among the NNs has also been maintained. In DEL, constructive strategy has been used for automatic determination of the number of NNs and constructive–pruning strategy has been used for automatic determination of the architecture of NNs in the ensemble. The hybrid constructive–pruning strategy has provided better diversity for the whole ensemble (Table 4b). NCL has been used for diversity of NNs in the ensemble encouraging individual networks to learn different regions and aspects of data space. But, if different NNs attempt to learn different regions with inaccurate architecture, learning will also be insufficient or improper by this attempt. Different training sets for individual networks are created which also help maintaining diversity among the NNs in the ensemble (Table 4b). In some cases, different training sets were

Table 7 Correlation of networks for the cancer dataset for $\eta = 0.1$, $\lambda = 1.0$

$Cor_{12} = (-0.006913)$	$Cor_{13} = 0.013130$	$Cor_{14} = (-0.017219)$
$Cor_{23} = (-0.006320)$	$Cor_{24} = (-0.005632)$	$Cor_{34} = (-0.005960)$

Table 8 Correlation of individual networks for the diabetes dataset for $\eta = 0.1$, $\lambda = 0.3$. In this case, iteration continued 212. In the ensemble, the number of individual networks required to complete training was 6

$Cor_{12} = 0.018343$	$Cor_{13} = 0.016651$	$Cor_{14} = 0.015793$
$Cor_{15} = 0.017334$	$Cor_{16} = 0.015708$	$Cor_{23} = 0.022907$
$Cor_{24} = 0.021727$	$Cor_{25} = 0.023847$	$Cor_{26} = 0.021609$
$Cor_{34} = 0.019722$	$Cor_{35} = 0.021647$	$Cor_{36} = 0.019616$
$Cor_{45} = 0.020531$	$Cor_{46} = 0.018605$	$Cor_{56} = 0.020420$

Table 9 Correlation of networks for the diabetes dataset for $\eta = 0.1$, $\lambda = 1.0$

$Cor_{12} = 0.024974$	$Cor_{13} = 0.006464$	$Cor_{14} = 0.000086$
$Cor_{15} = 0.029325$	$Cor_{16} = (-0.015724)$	$Cor_{23} = 0.007256$
$Cor_{24} = 0.000096$	$Cor_{25} = 0.032919$	$Cor_{26} = (-0.017651)$
$Cor_{34} = 0.000025$	$Cor_{35} = 0.008520$	$Cor_{36} = (-0.004569)$
$Cor_{45} = 0.000113$	$Cor_{46} = (-0.000060)$	$Cor_{56} = (-0.020727)$

created by variation of training examples, and in other cases by random choice of the training examples. As NN is a kind of unstable learning, random redistribution of the training samples has provided better learning in the case of an unstable learning. Both three- and four-layered individual networks were used to design the ensemble.

DEL uses a minimum number of parameters, i.e. only one correlation strength parameter λ. An incremental training approach has been used in DEL because even after choosing the appropriate architecture of the ensemble, DEL has to be trained several times for finding the correct value of the learning rate parameter and the correlation strength parameter λ. DEL uses only one cost function (the ensemble error E) during training, not two cost functions, one for accuracy and the one for diversity used in some other ensemble method in the literature. DEL uses a one-stage design process. Individual networks are created and combined at the same design stage. The advantage of DEL is that it does not need any separate gating block. DEL uses the parameter λ as a balancing mechanism for bias–variance–covariance tradeoff. Since DEL generates uncorrelated networks in the ensemble, individual networks in this ensemble are well diversified.

DEL algorithm uses both simple averaging and majority voting combination methods. For some problems, simple averaging method performed better, and for some other problems majority voting method performed better. Despite problem dependent, the choice of the correlation strength parameter λ is important in DEL. To delete hidden nodes from individual networks in an ensemble, initially a network larger than necessary is considered. But, assessing the initial size of the NN is challenging, which is still an unknown parameter in DEL algorithm.

6 Conclusions

DEL is a new algorithm for designing and training NN ensembles. Traditional way of ensemble designing is still a manual trial-and-error process, whereas DEL is an automatic design approach. The number of NNs and their architectures are determined by DEL algorithm.

The major benefits of the proposed DEL algorithm compared to existing ensemble algorithms are (1) automatic creation of ensemble architectures; (2) preservation of accuracy and diversity among the NNs in the ensemble; and (3) minimum number of parameters specified by designer.

DEL emphasizes both accuracy and diversity of NNs in ensemble to improve the performance. Constructive and constructive–pruning strategies are used in DEL to achieve the accuracy of individual NNs. To maintain diversity of NNs, NCL and different training sets are used. The performance of DEL algorithm was confirmed on benchmark problems. In almost all cases, DEL outperformed the others. However, the performance of DEL needs to be evaluated further on some regression and time series problems.

Compliance with ethical standards

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Li Y, Wei B, Liub Y, Yao L, Chena H, Yu J, Zhu W (2018) Incorporating knowledge into neural network for text representation. Expert Syst Appl 96:103–114
2. Hooshdar S, Adeli H (2004) Toward intelligent variable message signs in freeway work zones: a neural network model. J Transp Eng ASCE 130(1):83–93
3. Yu DL, Gomm JB (2002) Enhanced neural network modelling for a real multi-variable chemical process. Neural Comput Appl 10(4):289–299
4. Cengiz C, Köse E (2013) Modelling of color perception of different eye colors using artificial neural networks. Neural Comput Appl 23(7–8):2323–2332
5. Panakkat A, Adeli H (2007) Neural network models for earthquake magnitude prediction using multiple seismicity indicators. Int J Neural Syst 17(1):13–33
6. Ahmad Z, Zhang J (2005) Bayesian selective combination of multiple neural networks for improving long-range predictions in nonlinear process modelling. Neural Comput Appl 14(1):78–87
7. Tashakori AR, Adeli H (2002) Optimum design of cold-formed steel space structures using neural dynamic model. J Constr Steel Res 58(12):1545–1566
8. Gotsopoulos A, Saarimaki H, Glerean E, Jaaskelainen IP, Sams M, Nummenmaa L, Lampinen J (2018) Reproducibility of importance extraction methods in neural network-based fMRI classification. NeuroImage 181:44–54
9. Sá Junior JJM, Backes AR, Bruno OM (2018) Randomized neural network-based descriptors for shape classification. Neurocomputing 312:201–209
10. Vargas JAR, Pedrycz W, Hemerly EM (2019) Improved learning algorithm for two-layer neural networks for identification of nonlinear systems. Neurocomputing 329:86–96
11. Fourati F (2018) Multiple neural control and stabilization. Neural Comput Appl 29(12):1435–1442
12. Masulli F, Valentini G (2004) Effectiveness of error correcting output coding methods in ensemble and monolithic learning machines. Form Pattern Anal Appl 6(4):285–300
13. Srinivasana R, Wang C, Ho WK, Lim KW (2005) Neural network systems for multi-dimensional temporal pattern classification. Comput Chem Eng 29:965–981
14. Choudhury TA, Berndt CC, Man Z (2015) Modular implementation of artificial neural network in predicting in-flight particle characteristics of an atmospheric plasma spray process. Eng Appl Artif Intell 45:57–70
15. Sharkey NE, Sharkey AJ (1995) An analysis of catastrophic interference. Connect Sci 7:301–329
16. Gepperth A, Karaoguz C (2016) A bio-inspired incremental learning architecture for applied perceptual problems. Cogn Comput 8(5):924–934
17. McCloskey M, Cohen NJ (1989) Catastrophic interference in connectionist networks: the sequential learning problem. Psych Learn Motiv 24:109–165
18. French RM (1999) Catastrophic forgetting in connectionist networks. Trends Cogn Sci 3(4):128–135
19. Kemker R, McClure M, Abitino A, Hayes TL, Kanan C (2018) Measuring catastrophic forgetting in neural networks. In: The thirty-second AAAI conference on artificial intelligence (AAAI-18), February 2–7, 2018, New Orleans Riverside, New Orleans, LA, USA, pp 3390–3398
20. Robins A (1995) Catastrophic forgetting, rehearsal and pseudorehearsal. Connect Sci 7(2):123–146
21. Ren B, Wang H, Li J, Gao H (2017) Life-long learning based on dynamic combination model. Appl Soft Comput 56:398–404
22. Kirkpatrick J, Pascanu R, Rabinowitz N, Veness J, Desjardins G, Rusu AA, Milan K, Quan J, Ramalho T, Grabska-Barwinska A, Hassabis D, Clopath C, Kumaran D, Hadsell R (2017) Overcoming catastrophic forgetting in neural networks. Proc Natl Acad Sci 114(13):3521–3526
23. Coop R, Mishtal A, Arel I (2013) Ensemble learning in fixed expansion layer networks for mitigating catastrophic forgetting. IEEE Trans Neural Netw Learn Syst 24(10):1623–1634
24. Sharkey AJC (1996) On combining artificial neural nets. Connect Sci 8(3&4):299–313 **(special issue on combining artificial neural: ensemble approaches)**
25. Hansen LK, Salamon P (1990) Neural network ensembles. IEEE Trans Pattern Anal Mach Intell 12(10):993–1000
26. Granitto PM, Verdes PF, Ceccatto HA (2005) Neural network ensembles: evaluation of aggregation algorithms. Artif Intell 163:139–162
27. Jacobs RA (1997) Bias/variance analyses of mixtures-of-experts architectures. Neural Comput 9:369–383
28. Hancock T, Mamitsuka H (2012) Boosted network classifiers for local feature selection. IEEE Trans Neural Netw Learn Syst 23(11):1767–1778
29. Islam MM, Yao X, Nirjon SMS, Islam MA, Murase K (2008) Bagging and boosting negatively correlated neural networks. IEEE Trans Syst Man Cybern Part B Cybern 38(3):771–784
30. Rumelhart DE, Hinton GE, Williams RJ (1986) Learning representations by back-propagating errors. Nature 323(6088):533–536
31. Siddique NH, Tokhi MO (2001) Training neural networks: backpropagation vs genetic algorithms. In: Proceedings of the international joint conference on neural networks (IJCNN'01), 15–19 July 2001, Washington, DC, USA, pp 2673–2678
32. Liu Y, Yao X, Higuchi T (2000) Evolutionary ensembles with negative correlation learning. IEEE Trans Evol Comput 4:380–387
33. Chen H, Yao X (2010) Multiobjective neural network ensembles based on regularized negative correlation learning. IEEE Trans Knowl Data Eng 22(12):1738–1751
34. Liu Y, Yao X (1999) Ensemble learning via negative correlation. Neural Netw 12(10):1399–1404
35. Giacinto G, Roli F (2001) Design of effective neural network ensembles for image classification purposes. Image Vis Comput 19(9–10):699–707
36. Hashem S (1997) Optimal linear combinations of neural networks. Neural Netw 10(4):599–614
37. Tang EK, Suganthan PN, Yao X (2006) An analysis of diversity measures. Mach Learn 65(1):247–271
38. Brown G, Wyatt JL, Tino P (2005) Managing diversity in regression ensembles. J Mach Learn Res 6:1621–1650
39. Zhang ML, Zhou ZH (2013) Exploiting unlabeled data to enhance ensemble diversity. Data Min Knowl Discov 26(1):98–129
40. Rosen B (1996) Ensemble learning using de-correlated neural networks. Connect Sci 8(3–4):373–384 **(special issue on combining artificial neural: ensemble approaches)**
41. Liu Y, Yao X (1999) Simultaneous training of negatively correlated neural networks in an ensemble. IEEE Trans Syst Man Cybern B Cybern 29:716–725
42. Deb K, Agrawal S, Pratap A, Meyarivan T (2002) A fast and elitist multi-objective genetic algorithm: NSGA-II. IEEE Trans Evol Comput 6(2):182–197
43. Mousavi R, Eftekhari M (2015) A new ensemble learning methodology based on hybridization of classifier ensemble selection approaches. Appl Soft Comput 37:652–666
44. Islam MM, Yao X, Murase K (2003) A constructive algorithm for training cooperative neural network ensembles. IEEE Trans Neural Netw 14(4):820–834

45. Shahjahan M, Murase K (2006) A pruning algorithm for training cooperative neural network ensembles. IEICE Trans Inf Syst E89-D(3):1257–1269
46. Yin XC, Huang K, Hao HW (2015) DE^2: dynamic ensemble of ensembles for learning non-stationary data. Neurocomputing 165:14–22
47. Cruz RMO, Sabourin R, Cavalcanti GDC, Ren TI (2015) META-DES: a dynamic ensemble selection framework using meta-learning. Pattern Recogn 48:1925–1935
48. Chen H, Yao X (2009) Regularized negative correlation learning for neural network ensembles. IEEE Trans Neural Netw 20(12):1962–1979
49. Chen K, Wang S (2011) Semi-supervised learning via regularized boosting working on multiple semi-supervised assumptions. IEEE Trans Pattern Anal Mach Intell 33(1):129–143
50. Chen H, Jiang B, Yao X (2018) Semisupervised negative correlation learning. IEEE Trans Neural Netw Learn Syst 29(11):5366–5379
51. Soares RG, Chen H, Yao X (2017) A cluster-based semi-supervised ensemble for multiclass classification. IEEE Trans Emerg Top Comput Intell 1(6):408–420
52. Rafiei MH, Adeli H (2017) A new neural dynamic classification algorithm. IEEE Trans Neural Netw Learn Syst 28:12
53. Britto AS, Sabourin R, Oliveira LES (2014) Dynamic selection of classifiers—a comprehensive review. Pattern Recogn 47(11):3665–3680
54. Ren Y, Zhang L, Suganthan PN (2016) Ensemble classification and regression—recent developments, applications and future directions. IEEE Comput Intell Mag 11(1):41–53
55. Cruz RMO, Sabourin R, Cavalcanti GDC (2018) Dynamic classifier selection: recent advances and perspectives. Inf Fusion 41:195–216
56. Kwok TY, Yeung DY (1997) Constructive algorithms for structure learning in feed forward neural networks for regression problems. IEEE Trans Neural Netw 8:630–645
57. Ash T (1989) Dynamic node creation in backpropagation networks. Connect Sci 1(4):365–375
58. Reed R (1993) Pruning algorithms: a survey. IEEE Trans Neural Netw 4(5):740–747
59. Prechelt L (1998) Automatic early stopping using cross validation: quantifying the criteria. Neural Netw 11(4):761–767
60. Lichman M (2013) UCI machine learning repository. School of Information and Computer Science, University of California, Irvine, CA. http://archive.ics.uci.edu/ml
61. Kuncheva LI, Whitaker CJ (2003) Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy. Mach Learn 51:181–207
62. Opitz D, Maclin R (1999) Popular ensemble methods: an empirical study. J Artif Intell Res 11:169–198
63. Sharkey AJC, Sharkey NE (1997) Combining diverse neural nets. Connect Sci Knowl Eng Rev 12(3):231–247